

Lab 1 Report

Name: Rushat Rai

Student ID: 313591061

1. Training Results

Both variants use the same network (4×6 -tuples with 8 isomorphisms each), $\alpha = 0.1$ (0.1/32 per feature) and $P(2) = 0.7$, $P(4) = 0.3$, and were trained for 100,000 episodes with the submitted `td_state.cpp` and `td_afterstate.cpp`. Each point in Figure 1 is the mean score of 1,000 consecutive training episodes.



Figure 1. Mean score per 1,000 training episodes.

Table 1. Final results.

	TD-state	TD-after-state
Mean score, last 1,000 training episodes	39,063	42,672
2048 rate, last 1,000 training episodes	79.0%	79.9%
2048 rate, 1,000 games after loading the weights	81.2%	77.5%
Training time (single thread)	1 h 03 min	31 min

The two curves are almost identical for the first 40k episodes. TD-state is ahead between roughly 45k and 75k episodes, which fits its action selection averaging over every possible popup tile. It levels off near 39k towards the end, while TD-after-state keeps improving and finishes higher (mean 42.7k), mainly because it reaches 2048 more often. Both reach 2048 in about 80% of games. TD-state took about twice as long to train because it evaluates every possible next state for each move.

2. n-Tuple Network

There are far too many 2048 boards for a table of values, so V is approximated with an n -tuple network. The board is stored as 16 cells of 4 bits, each holding the tile exponent (0 = empty, 1 = 2, 2 = 4, ...). An n -tuple is a fixed set of n cells; the exponents in those cells, read as an n -digit base-16 number, index a lookup table of weights. Each 6-tuple therefore has a table of $16^6 \approx 16.8\text{M}$ weights, one per local tile configuration.

I use the four 6-tuples from the sample code: {0,1,2,3,4,5}, {4,5,6,7,8,9}, {0,1,2,4,5,6} and {4,5,6,8,9,10}. Because the value of a position does not change under rotation or reflection, each tuple is also applied to the 8 isomorphic placements (4 rotations $\times$ mirror), all sharing the same table. The value of a board is the sum of the $m = 4 \times 8 = 32$ looked-up weights:

$$V(b) = \sum_{i=1}^{32} \theta[\text{index}_i(b)]$$

This is linear in the weights, so a TD update with error δ adds $\alpha\delta/m = 0.1 \cdot \delta/32$ to each of the 32 weights used by the board (learning: :update divides by the 4 tuples, pattern: :update by the 8 isomorphisms). TD-state evaluates before-states (s and s'); TD-after-state evaluates after-states (s'').

3. TD(0), TD-state and TD-after-state

TD(0) learns a value function from experience by bootstrapping: after each step, the estimate is moved towards the reward plus the current estimate of the next state, without waiting for the end of the episode:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + V(S_{t+1}) - V(S_t)]$$

$R_{t+1} + V(S_{t+1})$ is the **TD-target** and $R_{t+1} + V(S_{t+1}) - V(S_t)$ is the **TD-error**. No discounting is used since the objective is the final score, and a terminal state has value 0. As in the assignment pseudocode, a whole episode is played first and the updates are then applied from the terminal step back to the first, so each target uses the successor's freshly updated value.

A move in 2048 splits into a deterministic part and a random part: $s \rightarrow (a, r) \rightarrow s' \rightarrow (\text{random tile}) \rightarrow s''$. The two variants learn values on different sides of the random tile:

TD-state learns $V(s)$ of before-states. $V(s)$ alone does not rank actions, so action selection must take the expectation over the random tile, which requires the popup model: $r + \sum P(s'')V(s'')$. The TD-target is $r + V(s'')$ for the s'' that actually occurred.

TD-after-state learns $V(s')$ of after-states. Since s' is a deterministic result of the move, action selection is just $r + V(s')$ with no expectation or model. The expectation over tiles is learned into $V(s')$ from samples. The TD-target is $r_{\text{next}} + V(s'_{\text{next}})$, where a_{next} is the greedy action in s'' .

4. Implementation

Both programs use the sample's game loop: each chosen move is stored in path as (s, s', a, r) , and s'' of a record is the before-state of the next record. After the episode, `update_episode` walks path from the terminal record back to the first.

4.1 TD-state

Action selection. For each legal action, `select_best_move` scores the move as $r + \text{expected_value}(s')$, which places a 2 (probability $0.7/k$) and a 4 (probability $0.3/k$) in each of the k empty cells of s' and sums the weighted $V(s'')$. A terminal s'' counts as 0; this can only happen when the popup fills the last empty cell.

$$\text{EVALUATE}(s, a) = r + \sum_s P(s, a, s'') V(s'')$$

```
float expected_value(const board& after) const {
    int empty[16], k = 0;
    for (int i = 0; i < 16; i++) {
        if (after.at(i) == 0) empty[k++] = i;
    }
    float expect = 0;
    for (int i = 0; i < k; i++) {
        board next = after;
        next.set(empty[i], 1);
        expect += 0.7f * value_of(next, k);
        next.set(empty[i], 2);
        expect += 0.3f * value_of(next, k);
    }
    return expect / k;
}
```

Learning. TD-target = $r + V(s'')$, TD-error = TD-target – $V(s)$. `v_next` holds $V(s'')$: it starts at 0 for the terminal state and is then set to the updated $V(s)$ of each step.

TD backup. The backup runs from the sampled next before-state s'' to s ; the after-state s' is only passed through and has no stored value.

$$s \xrightarrow{(a, r)} s' \xrightarrow{(\text{random tile})} s''$$

$$V(s) \leftarrow V(s) + \alpha [r + V(s'') - V(s)]$$

```
void update_episode(std::vector<state>& path, float alpha = 0.1) const {
    // V(s) <- V(s) + alpha * (r + V(s'') - V(s)), where V(terminal) = 0
    float v_next = 0;
    for (path.pop_back(); path.size(); path.pop_back()) {
        state& move = path.back();
        float td_target = move.reward() + v_next;
        float td_error = td_target - estimate(move.before_state());
        v_next = update(move.before_state(), alpha * td_error);
    }
}
```

4.2 TD-after-state

Action selection. Each legal action is scored as $r + V(s')$ using estimate on the after-state; no expectation over popup tiles.

$$\text{EVALUATE}(s, a) = r + V(s')$$

Learning. For each record, the greedy move in s'' is found again with the current weights, giving `a_next`, `r_next` and `s'_next`. TD-target = $r_{\text{next}} + V(s'_{\text{next}})$ (0 if s'' is terminal), TD-error = TD-target – $V(s')$.

TD backup. The backup runs from the next after-state s'_{next} to s' : the tile sampled after s' gives s'' , and the greedy action in s'' gives r_{next} and s'_{next} .

$$s' \xrightarrow{\text{(random tile)}} s'' \xrightarrow{\text{(a_next, r_next)}} s'_{next}$$

$$V(s') \leftarrow V(s') + \alpha [r_{next} + V(s'_{next}) - V(s')]$$

```
void update_episode(std::vector<state>& path, float alpha = 0.1) const {
    // V(s') <- V(s') + alpha * (r_next + V(s'_next) - V(s')), where a_next = argmax over A(s'')
    board next = path.back().before_state();
    for (path.pop_back(); path.size(); path.pop_back()) {
        state& move = path.back();
        state best = select_best_move(next);
        float td_target = best.is_valid() ? best.value() : 0;
        float td_error = td_target - estimate(move.after_state());
        update(move.after_state(), alpha * td_error);
        next = move.before_state();
    }
}
```